



UNIVERSITÀ DEGLI STUDI DI PERUGIA
Dipartimento di Ingegneria

CORSO DI LAUREA MAGISTRALE
INGEGNERIA INFORMATICA E ROBOTICA
DATA SCIENCE AND DATA ENGINEERING

PROGETTO

Signal Processing and Optimization for Big Data

-

**Model-Based Deep-Learning Approaches based on
ADMM for Robust PCA in Internet Traffic
Decomposition**

*From Pure Model-Based Optimization
to Adaptive Model-Based Deep Learning*

Dispensa di:
Daniele Angeloni

Professore:
Prof. Paolo Banelli

Anno Accademico 2024/2025

Indice

1	Introduzione	1
1.1	Contesto applicativo e motivazione	1
1.2	Framework RPCA: Low-rank plus sparse Model	2
1.3	Perché andare oltre il puro model-based: il ruolo del mismatch .	2
1.4	Model-Based Deep Learning	2
1.5	Obiettivi e organizzazione del lavoro	3
2	Formulazione Matematica del Problema	4
2.1	Low-Rank plus Sparse Model	4
2.2	Dal modello strutturale a RPCA	5
2.3	Risoluzione tramite ADMM	6
2.4	Aggiornamento di L : derivazione dell'operatore SVT	6
2.5	Aggiornamento di S : derivazione del soft-thresholding	7
2.6	Aggiornamento duale	8
2.7	Interpretazione strutturale	8
2.8	Complessità computazionale	8
3	Progettazione dei Dataset	9
3.1	Dataset Ideale (Matched Assumptions)	9
3.2	Dataset con Mismatch Strutturale	10
4	Approcci Implementati	12
4.1	RUN1 — ADMM su Dataset Ideale	13
4.2	RUN2 — ADMM su Dataset Mismatch	15
4.3	RUN3 — ADMM con Selezione Globale degli Iperparametri . .	16
4.4	RUN4 — Layer-wise Unfolded ADMM	18
4.5	RUN5 — Unfolded ADMM con Proximal Learnable	20
4.6	RUN6 — Hybrid Residual Unfolded ADMM	23
5	Analisi Comparativa e Discussione dei Risultati	27
5.1	Sintesi dei Risultati	27
5.2	Analisi della ricostruzione della componente low-rank	29
5.3	Analisi della ricostruzione della componente sparsa	31
5.4	Analisi della convergenza e dei valori singolari	33
5.5	Confronto Globale: Accuratezza ed Efficienza	36

INDICE	2
6 Conclusioni e Sviluppi Futuri	37
Riferimenti Bibliografici	39

Capitolo 1

Introduzione

1.1 Contesto applicativo e motivazione

La modellazione del traffico Internet costituisce un problema centrale nell'analisi dei sistemi di rete su larga scala. Le matrici di traffico descrivono l'evoluzione temporale dei volumi di comunicazione tra nodi o link e rappresentano uno strumento fondamentale per attività quali il capacity planning, l'anomaly detection, il traffic engineering e il network monitoring.

In molti scenari reali, il traffico osservato presenta una struttura fortemente correlata nel tempo e nello spazio. Questo comportamento è riconducibile alla presenza di pattern globali condivisi tra flussi, periodicità giornaliere o settimanali e dipendenze strutturali tra nodi della rete. Tali caratteristiche si traducono in una dinamica dominante di dimensione intrinseca ridotta, modellabile attraverso una componente a basso rango L_0 , che rappresenta il traffico nominale del sistema.

Parallelamente, possono verificarsi deviazioni localizzate nel tempo o nello spazio, associate a eventi non ordinari quali attacchi informatici, guasti, congestioni improvvise o burst anomali di traffico. Questi fenomeni tendono a coinvolgere un numero limitato di nodi o intervalli temporali e possono essere modellati tramite una componente strutturalmente distinta S_0 , caratterizzata da sparsità, nel senso che solo una frazione limitata degli elementi della matrice assume valori non nulli o significativamente elevati.

Alla luce di queste osservazioni, la matrice osservata viene modellata come

$$Y = L_0 + S_0 + N,$$

dove N rappresenta un termine di rumore additivo.

L'obiettivo del progetto è quindi la ricostruzione accurata delle componenti latenti L_0 e S_0 a partire da Y . La decomposizione del "*Low-rank plus sparse Model*" viene quindi studiata come problema di stima strutturata, con l'intento di separare in modo coerente la dinamica nominale globale dalle deviazioni

localizzate, analizzando la capacità del modello di rappresentare correttamente le strutture sottostanti ai dati osservati.

1.2 Framework RPCA: Low-rank plus sparse Model

Il problema rientra nel framework della decomposizione *low-rank plus sparse*, noto più in generale in letteratura anche come Robust Principal Component Analysis (RPCA). L'idea di base è che molte matrici di dati reali possano essere spiegate dalla sovrapposizione di una struttura di bassa dimensionalità (basso rango) e di una componente localizzata (sparsa o concentrata), e che la separazione delle due parti fornisca una descrizione compatta e robusta del fenomeno osservato.

RPCA fornisce una formulazione di riferimento per ottenere tale separazione attraverso un problema di ottimizzazione convesso (Principal Component Pursuit), che costituisce quindi la base teorica e metodologica. I dettagli matematici e teorici della formulazione saranno introdotti nel Capitolo 2.

1.3 Perché andare oltre il puro model-based: il ruolo del mismatch

Sebbene il framework RPCA offra una base solida, in applicazioni realistiche le ipotesi ideali che giustificano la separazione perfetta possono risultare solo approssimativamente soddisfatte. Ad esempio, il sottospazio associato alla dinamica nominale può variare nel tempo, la componente localizzata può assumere strutture persistenti (non puramente sparse elemento per elemento) e il livello di rumore può essere non trascurabile. In tali condizioni si parla di **model mismatch**: il modello rimane concettualmente valido, ma non descrive in modo esaustivo il processo sottostante ai dati osservati.

In presenza di mismatch, un approccio puramente basato sulle assunzioni del modello può degradare sensibilmente in termini di ricostruzione delle componenti, pur mantenendo correttezza formale rispetto alla formulazione di ottimizzazione. Questo fenomeno motiva l'esplorazione di strategie che preservino la struttura del modello, ma introducano flessibilità adattiva rispetto ai dati.

1.4 Model-Based Deep Learning

Nel presente lavoro il problema di decomposizione viene affrontato risolvendo la formulazione convessa di Principal Component Pursuit (PCP) tramite l'algoritmo di ottimizzazione Alternating Direction Method of Multipliers (ADMM). Quest'ultimo costituisce il riferimento model-based dello studio, in quanto

opera direttamente sulla formulazione deterministica derivata dalle assunzioni strutturali di rango basso e sparsità.

Tuttavia, quando le ipotesi teoriche alla base del modello non risultano pienamente soddisfatte, un approccio puramente ADMM-based può mostrare limiti in termini di capacità di adattamento ai dati reali. In tali situazioni emerge l'esigenza di integrare la struttura matematica dell'algoritmo con meccanismi di apprendimento dai dati.

Negli ultimi anni si è affermato il paradigma del **Model-Based Deep Learning**, che mira a combinare la solidità degli algoritmi di ottimizzazione con la flessibilità degli approcci di apprendimento. Nel contesto specifico di questo lavoro, ciò significa partire dalla struttura iterativa di ADMM e introdurre in modo controllato componenti apprendibili, mantenendo la struttura algoritmica originaria.

L'integrazione tra struttura algoritmica e apprendimento viene sviluppata in modo progressivo: dal semplice adattamento di parametri globali dell'ADMM, allo srotolamento dell'algoritmo in un'architettura a profondità finita tramite deep unfolding, fino all'introduzione di moduli parametrizzati in specifici passi iterativi. Questo percorso consente di mantenere la coerenza con la formulazione di ottimizzazione originaria, preservandone interpretabilità e stabilità, e al contempo di introdurre una flessibilità data-driven capace di mitigare il model mismatch.

1.5 Obiettivi e organizzazione del lavoro

L'obiettivo del progetto è analizzare e quantificare il beneficio dell'integrazione tra struttura algoritmica e apprendimento nel problema di decomposizione sopra descritto. A tal fine, vengono costruiti scenari controllati sia ideali sia in regime di model mismatch, valutando sistematicamente le prestazioni dell'approccio puramente ADMM-based e introducendo progressivamente componenti apprendibili, dall'ottimizzazione di iperparametri globali fino a configurazioni unfolded con moduli adattivi. L'analisi mira a individuare le condizioni in cui la formulazione basata solo sulle assunzioni del modello risulta sufficiente e quelle in cui invece l'integrazione data-driven diventa necessaria per compensare il mismatch.

La struttura del lavoro è la seguente:

- **Capitolo 2:** formulazione teorica e matematica del problema.
- **Capitolo 3:** costruzione dei dataset sperimentali.
- **Capitolo 4:** descrizione degli approcci, dalla baseline model-based alle varianti integrate con apprendimento.
- **Capitolo 5:** analisi dei risultati e dei principali trade-off.
- **Capitolo 6:** conclusioni e sviluppi futuri.

Capitolo 2

Formulazione Matematica del Problema

2.1 Low-Rank plus Sparse Model

Sia

$$Y \in \mathbb{R}^{N \times T}$$

una matrice osservata. Nel contesto del traffico Internet, N rappresenta il numero di nodi (o link) e T il numero di istanti temporali.

Il modello strutturale adottato assume che Y possa essere decomposta come

$$Y = L_0 + S_0 + N,$$

dove:

- L_0 è una matrice a basso rango,
- S_0 è una matrice sparsa,
- N è un termine di rumore additivo.

Questa rappresentazione è nota come *Low-Rank plus Sparse ($L+S$) Model* ed è alla base del framework di Robust Principal Component Analysis (RPCA), il cui obiettivo è stimare le componenti L_0 e S_0 a partire dalle osservazioni Y .

La struttura low-rank implica che:

$$\text{rank}(L_0) = r \ll \min(N, T),$$

ovvero L_0 può essere rappresentata tramite un numero ridotto di componenti latenti globali. La matrice sparsa S_0 soddisfa invece

$$\|S_0\|_0 \ll NT,$$

cioè contiene un numero limitato di elementi non nulli rispetto alla dimensione totale.

2.2 Dal modello strutturale a RPCA

Questo modello diventa, come già anticipato, un problema di **Robust Principal Component Analysis (RPCA)** quando si introduce un problema di ottimizzazione per stimare le due componenti.

Una formulazione naturale sarebbe:

$$\min_{L,S} \text{rank}(L) + \lambda \|S\|_0 \quad \text{s.t.} \quad Y = L + S.$$

Tuttavia:

- la funzione rango è non convessa,
- la norma ℓ_0 rende il problema combinatorio.

Il problema è quindi computazionalmente intrattabile in forma diretta.

Principal Component Pursuit (PCP)

Per ottenere una formulazione risolvibile, si introduce un rilassamento convesso noto come *Principal Component Pursuit (PCP)*:

$$\min_{L,S} \|L\|_* + \lambda \|S\|_1 \quad \text{s.t.} \quad Y = L + S.$$

dove:

- $\|L\|_* = \sum_i \sigma_i(L)$ è la norma nucleare (somma dei valori singolari),
- $\|S\|_1 = \sum_{i,j} |S_{ij}|$ è la norma ℓ_1 elemento per elemento.

La norma nucleare è il rilassamento convesso del rango, mentre la norma ℓ_1 è il rilassamento convesso della sparsità.

Il problema PCP è convesso e, sotto opportune ipotesi strutturali, consente il recupero esatto delle componenti originali. In particolare, è necessario che:

- la componente low-rank non sia troppo concentrata su poche coordinate (condizione di incoerenza);
- la componente sparsa sia sufficientemente rara;
- rango e livello di sparsità siano compatibili con la dimensione del problema.

In tali condizioni, la soluzione del problema convesso coincide con (L_0, S_0) con alta probabilità.

2.3 Risoluzione tramite ADMM

Per risolvere il problema PCP si adotta l'**Alternating Direction Method of Multipliers (ADMM)** nella sua forma scalata.

Consideriamo la formulazione vincolata:

$$\min_{L,S} \|L\|_* + \lambda \|S\|_1 \quad \text{s.t.} \quad Y - L - S = 0.$$

Lagrangiana aumentata (forma scalata)

Introducendo la variabile duale scalata U , la Lagrangiana aumentata può essere scritta come

$$\mathcal{L}_\rho(L, S, U) = \|L\|_* + \lambda \|S\|_1 + \frac{\rho}{2} \|Y - L - S + U\|_F^2 - \frac{\rho}{2} \|U\|_F^2,$$

dove $\rho > 0$ è il parametro di penalizzazione.

Aggiornamenti iterativi

ADMM alterna minimizzazioni rispetto alle variabili primali e un aggiornamento della variabile duale:

$$\begin{aligned} L^{k+1} &= \arg \min_L \|L\|_* + \frac{\rho}{2} \|Y - S^k + U^k - L\|_F^2, \\ S^{k+1} &= \arg \min_S \lambda \|S\|_1 + \frac{\rho}{2} \|Y - L^{k+1} + U^k - S\|_F^2, \\ U^{k+1} &= U^k + (Y - L^{k+1} - S^{k+1}). \end{aligned}$$

Questa formulazione, equivalente alla versione non scalata, risulta più compatta e numericamente conveniente, ed è quella adottata nel presente lavoro.

2.4 Aggiornamento di L : derivazione dell'operatore SVT

L'aggiornamento di L può essere riscritto come:

$$L^{k+1} = \arg \min_L \|L\|_* + \frac{\rho}{2} \|L - X^k\|_F^2,$$

dove

$$X^k = Y - S^k + U^k.$$

Questo problema coincide con l'operatore prossimale della norma nucleare:

$$L^{k+1} = \text{prox}_{(1/\rho)\|\cdot\|_*}(X^k).$$

Per derivarlo, si utilizza il fatto che sia la norma nucleare sia la norma di Frobenius sono invarianti per trasformazioni ortogonali. Si ha quindi la SVD di X^k :

$$X^k = U_X \Sigma_X V_X^\top,$$

con $\Sigma_X = \text{diag}(\sigma_i)$. Si può dimostrare che la soluzione ottima mantiene gli stessi vettori singolari e modifica soltanto i valori singolari, risolvendo:

$$\min_{\tilde{\sigma}_i \geq 0} \sum_i \tilde{\sigma}_i + \frac{\rho}{2} (\tilde{\sigma}_i - \sigma_i)^2.$$

Il problema è separabile per ciascun valore singolare, e la soluzione è:

$$\tilde{\sigma}_i = \max(\sigma_i - 1/\rho, 0).$$

Si ottiene quindi l'operatore di *Singular Value Thresholding*:

$$\text{SVT}_\tau(X) = U_X \text{diag}(\max(\sigma_i - \tau, 0)) V_X^\top.$$

In questo modo i valori singolari piccoli vengono annullati, promuovendo una soluzione a basso rango.

2.5 Aggiornamento di S : derivazione del soft-thresholding

L'aggiornamento di S è:

$$S^{k+1} = \arg \min_S \lambda \|S\|_1 + \frac{\rho}{2} \|S - Z^k\|_F^2,$$

dove

$$Z^k = Y - L^{k+1} + U^k.$$

Anche questo è un operatore prossimale:

$$S^{k+1} = \text{prox}_{(\lambda/\rho)\|\cdot\|_1}(Z^k).$$

Poiché la norma ℓ_1 è separabile elemento per elemento, il problema si riduce, per ciascun elemento scalare x , a:

$$\min_s \lambda |s| + \frac{\rho}{2} (s - x)^2.$$

La soluzione è ottenuta tramite condizioni di ottimalità e conduce alla regola di *soft-thresholding*:

$$\mathcal{S}_\tau(x) = \text{sign}(x) \max(|x| - \tau, 0), \quad \tau = \lambda/\rho.$$

Gli elementi con modulo inferiore alla soglia vengono annullati, mentre quelli più grandi vengono ridotti in ampiezza, promuovendo la sparsità della soluzione.

2.6 Aggiornamento duale

L'aggiornamento della variabile duale scalata è:

$$U^{k+1} = U^k + (Y - L^{k+1} - S^{k+1}).$$

Questo passo accumula il residuo primale e forza progressivamente il soddisfacimento del vincolo $Y \approx L + S$.

2.7 Interpretazione strutturale

Ogni iterazione ADMM può essere interpretata come la combinazione di:

- una proiezione verso matrici a basso rango (tramite SVT),
- una proiezione verso matrici sparse (tramite soft-thresholding),
- un aggiornamento duale che impone la coerenza con i dati osservati.

2.8 Complessità computazionale

L'operazione dominante per iterazione è la SVD richiesta dall'operatore SVT:

$$\mathcal{O}(NT \min(N, T)).$$

Questo costo può diventare significativo per matrici di grandi dimensioni, motivando l'introduzione, nei capitoli successivi, di varianti model-based deep learning (deep unfolding), in cui si limita il numero di iterazioni e si introducono parametri o operatori apprendibili.

Capitolo 3

Progettazione dei Dataset

Un aspetto fondamentale del progetto è la costruzione controllata di dataset sintetici, progettati non solo per testare le prestazioni numeriche degli algoritmi, ma per analizzarne il comportamento strutturale in presenza o meno delle ipotesi teoriche di Robust PCA.

La generazione dei dati è stata pensata quindi come uno strumento sperimentale per studiare il ruolo del *model mismatch*. A tal fine vengono costruiti due scenari distinti:

- un dataset ideale, in cui le assunzioni di RPCA sono pienamente soddisfatte;
- un dataset con mismatch strutturale, in cui tali assunzioni vengono violate in modo controllato.

Entrambi i dataset sono composti da 120 realizzazioni indipendenti di matrici $Y \in \mathbb{R}^{100 \times 500}$, suddivise in 80 campioni di training e 40 di test. Ogni matrice rappresenta il traffico osservato su 100 link nell'arco di circa due giorni, assumendo un ipotetico campionamento ogni 5 minuti.

3.1 Dataset Ideale (Matched Assumptions)

Nel dataset ideale si costruisce uno scenario perfettamente allineato alle ipotesi teoriche del Principal Component Pursuit.

La componente low-rank L_0 è generata tramite fattorizzazione bilineare:

$$L_0 = AB^\top,$$

con $A \in \mathbb{R}^{100 \times 5}$ e $B \in \mathbb{R}^{500 \times 5}$.

Il rango intrinseco è quindi fissato a $r = 5$. La matrice viene successivamente normalizzata in norma di Frobenius e scalata con fattore $L_{\text{scale}} = 10$.

La componente sparse S_0 viene generata con supporto casuale uniforme. Il livello di sparsità è fissato al 5% degli elementi totali:

$$\text{sparsity} = 0.05.$$

L'energia relativa è controllata imponendo:

$$\frac{\|S_0\|_F}{\|L_0\|_F} = 0.30.$$

Il rumore additivo è gaussiano e calibrato con:

$$\frac{\|N\|_F}{\|L_0\|_F} = 0.05.$$

Validazione statistica

La procedura di validazione ha restituito:

- Rango effettivo: 5.0 ± 0.0
- Sparsità: $5.0\% \pm 0.0\%$
- SNR medio: 26.4 dB
- Rapporto $\|S\|/\|L\|$: 0.300

Il dataset ideale soddisfa quindi in modo rigoroso le ipotesi teoriche di recovery esatto di PCP: sottospazio globale fisso, sparsità incoerente e rumore moderato.

3.2 Dataset con Mismatch Strutturale

Nel secondo scenario vengono introdotte violazioni controllate delle ipotesi strutturali di RPCA, al fine di studiare la robustezza degli algoritmi.

La componente low-rank non è più generata da un unico sottospazio globale, ma viene costruita in due segmenti temporali distinti, ciascuno con sottospazio differente. Inoltre, i coefficienti temporali non sono indipendenti tra loro, ma ogni valore dipende in larga parte da quello precedente (con un peso pari a 0.95). Questo significa che l'evoluzione nel tempo è molto regolare e cambia lentamente, introducendo una forte continuità temporale.

In aggiunta viene introdotta una perturbazione low-rank locale, attiva solo su un sottoinsieme di nodi (25% dei nodi) e con rango locale pari a 4. L'energia di questa componente locale rappresenta il 40% dell'energia totale, inducendo un aumento del rango effettivo globale.

Le anomalie non sono più sparse in modo indipendente elemento per elemento. Vengono generati 4 gruppi strutturati, ciascuno coinvolgente 6 nodi per una durata variabile tra 80 e 160 campioni temporali. Questo introduce correlazione spazio-temporale e rompe l'ipotesi di sparsità incoerente ed inoltre il rumore viene aumentato rispetto al dataset ideale.

Validazione statistica

La validazione del dataset mismatch evidenzia:

- Rango effettivo medio: ≈ 13
- Sparsità superiore rispetto al caso ideale (blocchi strutturati)
- SNR medio: ≈ 15 dB

L'aumento del rango effettivo è dovuto alla combinazione di:

- sottospazi tempo-varianti,
- perturbazioni locali,
- energia distribuita su più componenti latenti.

Questo scenario viola esplicitamente le condizioni teoriche di incoerenza e low-rank globale richieste per il recovery esatto di PCP. Tuttavia, le perturbazioni sono calibrate in modo controllato, così da non rendere il modello completamente inadeguato, ma piuttosto da simulare una situazione realistica di mismatch strutturale.

Discussione

Il confronto tra i due dataset evidenzia una transizione strutturale netta:

- Nel caso ideale, ADMM opera in un regime pienamente coerente con le ipotesi teoriche del modello low-rank plus sparse.
- Nel caso mismatch, tali ipotesi risultano solo parzialmente valide e il modello strutturale non descrive più perfettamente i dati.

Questa differenza costituisce la base sperimentale dell'intero progetto. Il dataset ideale consente di valutare il comportamento dell'algoritmo in condizioni teoricamente favorevoli, mentre il dataset mismatch permette di analizzarne la robustezza quando le assunzioni vengono violate in modo controllato.

Su questa base si inserisce la progressiva introduzione di componenti apprendibili all'interno della struttura ADMM, con l'obiettivo di studiare se e in che misura l'integrazione tra modello e apprendimento possa compensare le imperfezioni del modello strutturale.

Capitolo 4

Approcci Implementati

In questo capitolo vengono descritti gli approcci sperimentali sviluppati per la decomposizione di matrici di traffico secondo il modello a basso rango più componente sparsa.

L'attenzione non è rivolta alla semplice applicazione di un algoritmo standard, ma alla costruzione di una progressione metodologica controllata, che consente di analizzare in modo sistematico l'effetto dell'introduzione dell'apprendimento all'interno di una struttura algoritmica model-based.

Tutti gli esperimenti condividono lo stesso modello di riferimento e le stesse metriche di valutazione; ciò che varia è il grado di adattività introdotto rispetto alla formulazione ADMM classica.

La sequenza degli approcci implementati è la seguente:

- **RUN1 — ADMM (Ideal):** algoritmo ADMM classico applicato al dataset ideale, coerente con le assunzioni teoriche del modello.
- **RUN2 — ADMM (Mismatch):** stessa configurazione di RUN1 applicata però al dataset con mismatch strutturale, per valutare la sensibilità del metodo e creare una baseline.
- **RUN3 — Global-Learned ADMM:** selezione data-driven globale degli iperparametri λ e ρ , mantenendo invariata la struttura algoritmica.
- **RUN4 — Layer-wise Unfolded ADMM:** versione unfoldata di ADMM con iper-parametri specifici e variabili per layer apprendibili tramite training supervisionato.
- **RUN5 — Unfolded ADMM con Proximal Learnable:** sostituzione del proximal operator della componente sparse con una rete neurale, mantenendo invariato l'aggiornamento low-rank.
- **RUN6 — Unfolded Hybrid Residual ADMM:** architettura ibrida in cui entrambe le componenti (low-rank e sparse) sono corrette tramite blocchi residui learnable, preservando la struttura Unfolded ADMM.

Metriche di Valutazione

Le prestazioni vengono valutate principalmente in termini di accuratezza di ricostruzione delle componenti latenti:

$$\text{NMSE}_L = \frac{\|L_0 - \hat{L}\|_F^2}{\|L_0\|_F^2},$$

$$\text{NMSE}_S = \frac{\|S_0 - \hat{S}\|_F^2}{\|S_0\|_F^2},$$

dove $\hat{L}$ e $\hat{S}$ indicano le stime prodotte dai vari approcci.

Oltre all'accuratezza di ricostruzione vengono analizzate:

- la velocità di convergenza (numero di iterazioni o layer);
- la stabilità delle soluzioni in presenza di mismatch;
- il compromesso tra complessità computazionale e capacità adattiva.

Il confronto tra i diversi RUN consente quindi di studiare in modo controllato il trade-off tra rigidità strutturale del modello e flessibilità introdotta dall'apprendimento, mantenendo costante il problema di base.

4.1 RUN1 — ADMM su Dataset Ideale

Il primo esperimento consiste nell'applicazione dell'algoritmo ADMM classico al dataset ideale, costruito in modo coerente con le ipotesi teoriche di recovery del problema convesso di Principal Component Pursuit, rilassamento di RPCA, dove non viene introdotto alcun meccanismo di apprendimento in questa configurazione. Questo approccio serve quindi a darci informazioni su quanto è qualitativamente corretto l'uso di ADMM per il problema PCP in caso di assenza di mismatch e quindi sfruttando le pure assunzioni sul modello ipotizzato.

Configurazione

Il parametro di regolarizzazione λ è impostato secondo la scelta teorica suggerita in letteratura (da Candès et al.) per il problema di Principal Component Pursuit, ossia $\lambda = 1/\sqrt{\max(N, T)}$, che garantisce condizioni di recovery esatto sotto opportune ipotesi strutturali.

Il parametro algoritmico ρ di ADMM è invece scelto empiricamente (in questo caso 1) e può essere adattato per migliorare la convergenza.

$$\lambda = \frac{1}{\sqrt{\max(N, T)}}, \quad \rho = 1.$$

Con $N = 100$ e $T = 500$, si ottiene:

$$\lambda \approx 0.0447.$$

Risultati sperimentali

L'algoritmo è stato valutato su 40 matrici di test di dimensione 100×500 . Le metriche medie ottenute sono:

- $\text{NMSE}_L = 0.000580 \pm 0.000012$,
- $\text{NMSE}_S = 0.023465 \pm 0.00015$,
- Numero medio di iterazioni: 432.9 ± 3.5 .

L'errore sulla componente low-rank risulta estremamente basso, con NMSE inferiore a 10^{-3} , indicando che, in condizioni molto coerenti con le ipotesi del modello, ADMM è in grado di ricostruire con elevata precisione il sottospazio nominale.

La componente sparse presenta un errore maggiore rispetto alla componente low-rank, ma comunque contenuto, coerentemente con la presenza di rumore e con l'effetto di shrinkage introdotto dal soft-thresholding.

Analisi del comportamento

I risultati confermano le previsioni teoriche di RPCA nel contesto ideale:

- ADMM recupera in modo accurato la struttura low-rank globale.
- La varianza delle metriche è estremamente ridotta, evidenziando stabilità numerica.
- Il numero di iterazioni, pari a circa 433, riflette una convergenza stabile ma relativamente lenta, tipica degli algoritmi di ottimizzazione convessa basati su splitting.

Significato di RUN1

RUN1 rappresenta quindi il riferimento superiore teorico per un approccio puramente basato sulle assunzioni del modello e costituisce una base solida rispetto alla quale verranno confrontate le successive configurazioni, in presenza però di mismatch strutturale.

4.2 RUN2 — ADMM su Dataset Mismatch

Nel secondo esperimento viene applicata la stessa configurazione ADMM utilizzata in RUN1 al dataset con mismatch strutturale.

L'algoritmo assume ancora il *Low-Rank plus Sparse Model*:

$$Y = L + S,$$

ma in questo caso le ipotesi strutturali alla base di Principal Component Pursuit non sono più soddisfatte. In particolare:

- il sottospazio low-rank è tempo-variante,
- sono presenti perturbazioni low-rank locali,
- le anomalie sono strutturate e persistenti nel tempo,
- il livello di rumore è significativamente più elevato ($\text{SNR} \approx 15$ dB).

Non viene introdotto alcun meccanismo di apprendimento: l'algoritmo ADMM è quindi identico a RUN1. Questo esperimento misura quindi la degradazione di un approccio basato puramente sulle assunzioni del modello, in presenza però di model mismatch.

Configurazione

Gli iper-parametri restano invariati rispetto al caso ideale, consentendo un confronto corretto:

$$\lambda = \frac{1}{\sqrt{\max(N, T)}}, \quad \rho = 1.$$

Risultati sperimentali

L'algoritmo è stato valutato su 40 matrici di test di dimensione 100×500 .

Le metriche medie ottenute sono:

- $\text{NMSE}_L = 0.053764 \pm 0.005426$,
- $\text{NMSE}_S = 0.822088 \pm 0.057970$,
- Numero medio di iterazioni: 187.8 ± 2.6 .

Rispetto a RUN1 si osserva un peggioramento drastico delle prestazioni, in particolare sulla componente sparse.

Analisi del comportamento

Il confronto diretto con RUN1 evidenzia una forte degradazione:

- L'errore sulla componente low-rank aumenta di circa due ordini di grandezza.
- L'NMSE sulla componente sparse si avvicina a 0.8, indicando una ricostruzione fortemente deteriorata.

Questa degradazione è coerente con le violazioni strutturali introdotte nel dataset mismatch:

- La variazione del sottospazio nel tempo impedisce una rappresentazione low-rank globale coerente.
- Le perturbazioni locali aumentano il rango effettivo (osservato intorno a 13).
- Le anomalie strutturate non sono ben modellate dalla sola regolarizzazione ℓ_1 elemento-wise.

Un aspetto interessante è la riduzione del numero medio di iterazioni (da circa 433 a 188). Questo non indica una convergenza migliore, ma piuttosto una convergenza verso una soluzione subottima coerente con il modello assunto, che tuttavia non è adeguato a descrivere la struttura reale dei dati.

Significato di RUN2

RUN2 rappresenta il punto critico del progetto e quindi si pone come la baseline da migliorare: mostra chiaramente che un approccio model-based, pur teoricamente fondato, può fallire in modo significativo quando le assunzioni strutturali vengono violate.

Questo risultato motiva l'introduzione progressiva di meccanismi di apprendimento nei successivi RUN, con l'obiettivo di compensare il mismatch tra modello e dati.

4.3 RUN3 — ADMM con Selezione Globale degli Iperparametri

Nel terzo esperimento viene introdotto un primo livello di adattamento ai dati, mantenendo invariata la struttura iterativa dell'algoritmo ADMM.

A differenza di RUN2, in cui i parametri erano fissati a priori, in RUN3 i parametri di regolarizzazione λ e ρ vengono selezionati tramite una procedura di ottimizzazione offline su un sottoinsieme del dataset di training.

Non vengono ancora introdotti unfolding, parametri layer-wise o operatori learnable. L'unica modifica consiste nella scelta data-driven degli iperparametri globali.

Procedura di selezione tramite *GridSearch*

La selezione, tramite *GridSearch*, è stata effettuata su un sottoinsieme del training set, per abbassare i tempi computazionali elevati.

Si è esplorata una griglia di valori:

$$\lambda \in \lambda_0 \cdot \{0.25, 0.5, 0.75, 1, 1.25, 1.5, 2\},$$

$$\rho \in \{0.5, 0.75, 1.0, 1.25, 1.5, 2.0\},$$

dove $\lambda_0 = 1/\sqrt{\max(N, T)}$ rappresenta il valore teorico proposto per Principal Component Pursuit.

Per ogni coppia (λ, ρ) è stato calcolato uno score congiunto sulle due componenti:

$$\text{score} = \sqrt{\text{NMSE}_L \cdot \text{NMSE}_S}.$$

La combinazione che minimizza lo score medio è risultata:

$$\lambda_{\text{best}} = 0.044721, \quad \rho_{\text{best}} = 2.0.$$

Si osserva che il valore ottimale di λ coincide con quello teorico, mentre il parametro ρ viene raddoppiato rispetto alla configurazione baseline.

Risultati sperimentali

Con i parametri selezionati, l'algoritmo è stato valutato sull'intero test set e le metriche medie ottenute sono:

- $\text{NMSE}_L = 0.053762 \pm 0.005426$,
- $\text{NMSE}_S = 0.822068 \pm 0.057970$,
- Numero medio di iterazioni: 94.9 ± 1.4 .

Analisi del comportamento

Il confronto con RUN2 evidenzia un aspetto rilevante.

Le metriche di ricostruzione rimangono sostanzialmente invariate: l'errore su L e su S non mostra miglioramenti significativi. Questo indica che la semplice ottimizzazione globale degli iperparametri non è sufficiente a compensare le violazioni strutturali presenti nel dataset mismatch.

Il parametro λ controlla il bilanciamento tra promozione del basso rango e promozione della sparsità: valori maggiori penalizzano maggiormente la componente S , mentre valori più piccoli consentono una maggiore energia nella parte sparsa. Il fatto che il valore ottimale coincida con quello teorico

suggerisce che il problema non risieda in un errato bilanciamento tra le due componenti, ma nella struttura stessa del modello.

Il parametro ρ , invece, non modifica la soluzione ottima del problema convesso, ma influenza la dinamica di convergenza di ADMM. Esso regola il peso del termine di penalizzazione quadratica nella Lagrangiana aumentata e quindi la velocità con cui viene imposto il vincolo di consistenza $Y \approx L + S$.

Infatti, si osserva una riduzione marcata del numero medio di iterazioni, che passa da circa 188 a circa 95. L'aumento di ρ accelera il soddisfacimento del vincolo primale, migliorando la velocità di convergenza senza modificare la qualità finale della soluzione.

Significato di RUN3

RUN3 rappresenta una prima integrazione tra modello e dati, limitata però alla scelta degli iperparametri globali.

I risultati mostrano che il limite principale non risiede tanto nella scelta di λ e ρ , quanto nella rigidità strutturale del modello low-rank + sparse stesso.

Questo passaggio evidenzia che, per affrontare efficacemente il model mismatch, non è sufficiente adattare parametri scalari globali, ma occorre intervenire direttamente sulla dinamica iterativa dell'algoritmo, come verrà fatto nei RUN successivi tramite deep unfolding.

4.4 RUN4 — Layer-wise Unfolded ADMM

Nel quarto esperimento viene introdotto il paradigma del *deep unfolding* applicato ad ADMM. L'algoritmo non viene più eseguito fino a convergenza, ma troncato a un numero finito di iterazioni K , ciascuna interpretata come un layer di una rete neurale profonda.

La struttura dell'algoritmo rimane invariata, ma i parametri diventano dipendenti dal layer e vengono appresi tramite backpropagation sul dataset di training. In questo modo si mantiene l'interpretabilità dell'algoritmo ADMM, introducendo al contempo una dinamica adattiva guidata dai dati.

Configurazione

Nel presente esperimento si fissa:

$$K = 10.$$

Per ciascun layer k vengono introdotti parametri learnable λ_k e ρ_k , che sostituiscono gli iper-parametri globali delle versioni precedenti.

Gli aggiornamenti mantengono la struttura scaled-ADMM:

$$\begin{aligned}
L^{k+1} &= \text{SVT}_{1/\rho_k}(Y - S^k + U^k), \\
S^{k+1} &= \mathcal{S}_{\lambda_k/\rho_k}(Y - L^{k+1} + U^k), \\
U^{k+1} &= U^k + (Y - L^{k+1} - S^{k+1}).
\end{aligned}$$

I proximal operator restano invariati:

- **SVT** per la promozione del basso rango,
- **soft-thresholding** per la promozione della sparsità.

Per garantire che i parametri restino positivi durante l'addestramento, essi non vengono ottimizzati direttamente, ma parametrizzati in forma logaritmica. In questo modo si esegue un'ottimizzazione non vincolata sui parametri interni θ_k e ϕ_k , mentre i valori effettivi utilizzati dall'algoritmo sono sempre strettamente positivi:

$$\lambda_k = \exp(\theta_k), \quad \rho_k = \exp(\phi_k).$$

Il modello viene addestrato sul dataset mismatch utilizzando:

- ottimizzatore Adam,
- learning rate 5×10^{-4} ,
- early stopping con patience pari a 10 epoche.

La funzione obiettivo è definita come:

$$\mathcal{L} = \text{NMSE}_L + \text{NMSE}_S,$$

calcolata rispetto alle componenti di ground truth.

La loss di validazione decresce in modo monotono e regolare, passando da circa 0.93 a circa 0.47. Questo comportamento suggerisce una dinamica di apprendimento stabile e ben condizionata, senza fenomeni di divergenza o degradazione della generalizzazione.

Risultati sperimentali

Valutando il modello addestrato sull'intero set di test mismatch si ottengono:

- $\text{NMSE}_L = 0.052066$,
- $\text{NMSE}_S = 0.425005$,
- Numero di layer: $K = 10$.

Analisi del comportamento

Rispetto alla baseline RUN2, si osservano due effetti distinti.

- L'errore sulla componente low-rank rimane sostanzialmente invariato.
- L'NMSE sulla componente sparse si riduce in modo significativo (da circa 0.82 a circa 0.43).

Inoltre, questo risultato è ottenuto tramite soli 10 layer, invece che circa 188 iterazioni (RUN2), considerando comunque il processo di addestramento iniziale che avviene però una sola volta.

Il miglioramento sulla componente sparsa può essere interpretato come conseguenza della dinamica layer-wise dei parametri λ_k e ρ_k . A differenza della versione classica, in cui la soglia λ/ρ è costante, qui la soglia varia lungo la traiettoria iterativa, permettendo una modulazione progressiva della penalizzazione della sparsità.

Tuttavia, poiché gli operatori prossimali restano invariati, la struttura del modello convesso non viene modificata. Di conseguenza, il miglioramento risulta limitato soprattutto alla componente sparse, mentre la parte low-rank continua a risentire del mismatch strutturale del dataset.

Significato di RUN4

RUN4 rappresenta il primo passo sostanziale oltre il modello puramente statico.

La struttura ADMM viene preservata, garantendo interpretabilità e coerenza con il problema convesso originale, ma la dinamica dei parametri diventa data-driven. Questo consente un miglior adattamento alla distribuzione statistica osservata, migliorando sensibilmente le prestazioni sulla componente sparsa e riducendo drasticamente il numero di iterazioni necessarie.

Tuttavia, l'assenza di modifiche agli operatori prossimali limita la capacità del modello di compensare completamente il mismatch strutturale, motivando l'introduzione di operatori learnable nei RUN successivi.

4.5 RUN5 — Unfolded ADMM con Proximal Learnable

Nel quinto approccio viene introdotto un ulteriore livello di flessibilità all'interno dell'architettura unfolded. Come in RUN4, l'algoritmo ADMM viene interpretato come una rete a $K = 10$ layer. Tuttavia, in questo caso il proximal operator associato alla componente sparse non è più il soft-thresholding analitico, ma viene sostituito da un operatore parametrico learnable.

La struttura globale dell'algoritmo quindi invariata, ma l'aggiornamento di S diventa addestrabile sui dati.

Struttura dell'architettura

Per ciascun layer k , gli aggiornamenti assumono la forma:

$$\begin{aligned} L^{k+1} &= \text{SVT}_{1/\rho_k}(Y - S^k + U^k), \\ S^{k+1} &= \mathcal{P}_{\theta_k}(Y - L^{k+1} + U^k), \\ U^{k+1} &= U^k + (Y - L^{k+1} - S^{k+1}), \end{aligned}$$

dove:

- SVT rimane il proximal analitico della norma nucleare,
- $\mathcal{P}_{\theta_k}$ è un operatore learnable che sostituisce il soft-thresholding,
- λ_k e ρ_k sono parametri layer-wise learnable.

L'operatore $\mathcal{P}_{\theta_k}$ è implementato come una rete neurale convoluzionale compatta applicata alla matrice Z interpretata come immagine bidimensionale (nodi $\times$ tempo).

La struttura del blocco è la seguente:

- convoluzione 3×3 con 32 canali,
- attivazione ReLU,
- seconda convoluzione 3×3 con 32 canali,
- attivazione ReLU,
- convoluzione finale 3×3 che riporta a un singolo canale.

Il blocco produce una correzione residua scalata:

$$\mathcal{P}_{\theta_k}(Z) = Z - \alpha_k f_{\theta_k}(Z),$$

dove:

- f_{θ_k} è la rete convoluzionale descritta sopra,
- α_k è un parametro scalare learnable che controlla l'intensità della correzione.

La scelta di convoluzioni 3×3 con padding unitario permette di catturare dipendenze locali nello spazio nodi-tempo, modellando correlazioni strutturate tra nodi vicini e tra istanti temporali adiacenti, senza introdurre un numero eccessivo di parametri.

La formulazione residua garantisce stabilità: in assenza di necessità di correzione, il blocco può approssimare facilmente l'identità, preservando la dinamica ADMM originale.

A differenza del soft-thresholding, che impone sparsità elemento per elemento, questo operatore può modellare dipendenze locali e strutture spaziali, risultando più adatto a descrivere anomalie persistenti e correlate.

Addestramento

Il modello viene addestrato sul dataset mismatch utilizzando l'ottimizzatore Adam con learning rate pari a 5×10^{-4} ed early stopping con patience pari a 10 epoche.

La funzione obiettivo è:

$$\mathcal{L} = \text{NMSE}_L + \text{NMSE}_S.$$

Durante l'addestramento si osserva una riduzione marcata della loss di validazione, che passa da valori iniziali superiori a 10 fino a circa 0.33, indicando un apprendimento efficace e numericamente stabile.

Risultati sperimentali

Valutando il modello addestrato sull'intero set di test mismatch, si ottengono:

- $\text{NMSE}_L = 0.0356$,
- $\text{NMSE}_S = 0.2823$,
- Numero di layer: $K = 10$.

Analisi del comportamento

Rispetto a RUN4 si osserva un miglioramento significativo su entrambe le componenti:

- L'errore sulla componente low-rank si riduce da circa 0.052 a 0.036.
- L'errore sulla componente sparse si riduce da circa 0.43 a circa 0.28.

Il miglioramento marcato sulla componente sparse evidenzia che il soft-thresholding classico non è sufficiente a modellare anomalie strutturate e persistenti. L'operatore learnable riesce invece a catturare correlazioni locali e pattern spazio-temporali non rappresentabili tramite una penalizzazione ℓ_1 elemento per elemento.

Significato di RUN5

RUN5 rappresenta un'estensione sostanziale dell'approccio unfolded. La struttura iterativa di ADMM viene preservata, ma il proximal operator della componente sparse non è più analitico: viene appreso dai dati tramite una rete convoluzionale.

Questa configurazione può essere interpretata come una forma di *DNN-aided optimization*: l'algoritmo resta guidato dal modello, ma uno dei suoi blocchi fondamentali viene sostituito da un operatore neurale parametrico.

L'impostazione è inoltre affine al paradigma *Plug-and-Play*, in cui un operatore learnable rimpiazza una regolarizzazione esplicita mantenendo la struttura iterativa originale.

Rispetto a RUN4 l'aumento di espressività è evidente: il soft-thresholding impone sparsità elemento per elemento, mentre l'operatore convoluzionale può catturare correlazioni locali e strutture spazio-temporali più complesse.

Di contro, l'algoritmo non può più essere interpretato come un metodo di ottimizzazione convessa con garanzie teoriche formali. RUN5 si colloca quindi in una posizione intermedia: più flessibile del modello puro, ma ancora strutturato attorno alla dinamica di ADMM.

4.6 RUN6 — Hybrid Residual Unfolded ADMM

Nel sesto approccio viene introdotta l'architettura più espressiva dell'intero progetto. L'algoritmo ADMM viene unfoldato in $K = 10$ layer, come nei RUN precedenti, ma in questo caso sia l'aggiornamento della componente low-rank sia quello della componente sparse vengono arricchiti con correzioni residue learnable.

L'idea è preservare completamente la struttura analitica di ADMM (SVT e soft-thresholding), ma permettere al modello di apprendere deviazioni sistematiche rispetto al comportamento imposto dal modello assunto e quindi compensare il più possibile il mismatch.

Struttura dell'architettura

Per ciascun layer k vengono eseguiti dapprima gli aggiornamenti analitici standard, seguiti da correzioni additive residue rispetto all'aggiornamento analitico:

$$\begin{aligned} L_{\text{svt}}^{k+1} &= \text{SVT}_{1/\rho_k}(Y - S^k + U^k), \\ L^{k+1} &= L_{\text{svt}}^{k+1} + \mathcal{R}_{\theta_k}^L(L_{\text{svt}}^{k+1}), \\ S_{\text{soft}}^{k+1} &= \mathcal{S}_{\lambda_k/\rho_k}(Y - L^{k+1} + U^k), \\ S^{k+1} &= S_{\text{soft}}^{k+1} + \mathcal{R}_{\phi_k}^S(S_{\text{soft}}^{k+1}), \\ U^{k+1} &= U^k + (Y - L^{k+1} - S^{k+1}). \end{aligned}$$

dove:

- SVT è il proximal operator della norma nucleare,
- $\mathcal{S}$ è l'operatore di soft-thresholding,
- $\mathcal{R}_{\theta_k}^L$ e $\mathcal{R}_{\phi_k}^S$ sono blocchi convoluzionali residuali learnable.

I parametri λ_k e ρ_k rimangono layer-wise learnable e vengono parametrizzati in forma esponenziale per garantirne la positività.

Struttura dei blocchi convoluzionali

Le correzioni residue $\mathcal{R}_k^L$ e $\mathcal{R}_k^S$ sono implementate tramite blocchi convoluzionali compatti, condivisi nella struttura ma con parametri distinti per ciascun layer.

Ciascun blocco è costituito da:

- una convoluzione 3×3 con 1 canale in ingresso e 32 canali intermedi,
- funzione di attivazione ReLU,
- una seconda convoluzione 3×3 con 32 canali,
- funzione di attivazione ReLU,
- una terza convoluzione 3×3 che riporta il numero di canali a 1.

Tutte le convoluzioni utilizzano *padding* unitario, in modo da preservare la dimensione spaziale della matrice (nodi $\times$ tempo).

L'uscita del blocco viene inoltre scalata da un parametro learnable α_k inizializzato a 0.05:

$$\mathcal{R}_k(X) = \alpha_k f_{\theta_k}(X),$$

dove f_{θ_k} rappresenta la trasformazione convoluzionale. La presenza del fattore scalare α_k stabilizza il training nelle fasi iniziali, limitando l'ampiezza della correzione residua.

La correzione viene poi applicata in forma residuale:

$$X^{k+1} = X_{\text{base}}^{k+1} + \mathcal{R}_k(X_{\text{base}}^{k+1}),$$

dove X_{base}^{k+1} rappresenta l'aggiornamento ottenuto tramite l'operatore (SVT per L , soft-thresholding per S).

L'uso di convoluzioni 3×3 consente di:

- catturare dipendenze locali tra nodi e istanti temporali adiacenti,
- modellare strutture spaziali e temporali persistenti,
- mantenere un numero di parametri contenuto rispetto a strati fully-connected.

Poiché la correzione è aggiunta in forma residua, in assenza di mismatch il blocco può approssimare facilmente la trasformazione identità. Questo preserva la stabilità della dinamica ADMM e mantiene l'interpretabilità della struttura model-based originale.

Addestramento

Il modello viene addestrato sul dataset di training mismatch utilizzando l'ottimizzatore Adam con learning rate pari a 5×10^{-4} . È impiegata una strategia di early stopping con patience pari a 10 epoche.

La funzione obiettivo è definita come:

$$\mathcal{L} = \text{NMSE}_L + \text{NMSE}_S,$$

calcolata tra le componenti ricostruite e quelle di ground truth.

Durante l'addestramento si osserva una riduzione progressiva e stabile della loss di validazione, che passa da circa 0.79 a valori prossimi a 0.13. L'andamento monotonicamente decrescente e l'assenza di oscillazioni significative indicano un processo di apprendimento stabile, senza segnali evidenti di instabilità numerica o overfitting precoce.

Risultati sperimentali

Valutando il modello addestrato sull'intero set di test mismatch si ottengono:

- $\text{NMSE}_L = 0.0191$,
- $\text{NMSE}_S = 0.1210$,
- Numero di layer: $K = 10$.

Analisi del comportamento

Rispetto ai RUN precedenti si osserva un miglioramento marcato su entrambe le componenti:

- L'errore sulla componente low-rank si riduce significativamente rispetto a RUN4 e RUN5.
- L'errore sulla componente sparse scende da circa 0.28 (RUN5) a circa 0.12.

Il miglioramento sulla componente low-rank evidenzia che il solo SVT non è sufficiente a modellare la natura tempo-variante e localmente perturbata del sottospazio nel dataset mismatch. Le correzioni residue permettono di adattare dinamicamente la stima del sottospazio, superando la rigidità della penalizzazione nucleare pura.

Il miglioramento osservato sulla componente sparse, nonostante l'aggiornamento di S fosse già stato reso learnable in RUN5, evidenzia l'accoppiamento strutturale tra le due variabili (L e S).

Infatti, l'aggiornamento di S dipende direttamente dalla stima corrente di L . Una stima più accurata del sottospazio low-rank riduce la contaminazione

nell'input del proximal sparse, consentendo una separazione più efficace tra le due componenti.

In questo senso, le correzioni residue applicate anche alla componente L non migliorano soltanto la ricostruzione low-rank, ma stabilizzano l'intero processo di decomposizione, portando a un miglioramento congiunto di entrambe le metriche.

Significato di RUN6

RUN6 rappresenta la configurazione più espressiva del framework di **Model-Based Deep Learning** basato su deep unfolding.

L'algoritmo ADMM viene srotolato (unfolded) in una rete neurale a profondità finita di $K = 10$ layer, dove ciascun layer corrisponde a un'iterazione con parametri e operatori apprendibili.

L'unfolding introduce tre elementi principali:

- un numero fisso di iterazioni K , che determina la profondità della rete;
- parametri layer-wise (λ_k, ρ_k) , che consentono una dinamica adattiva lungo le iterazioni;
- operatori correttivi learnable, che correggono in maniera ottimale gli aggiornamenti basati sui proximal operator.

La struttura dell'ADMM viene infatti comunque preservata (SVT per la componente low-rank, soft-thresholding per la componente sparse, aggiornamento duale), ma ogni aggiornamento è arricchito dalla correzione residua appresa dai dati.

In questo modo il modello mantiene l'interpretabilità del framework convesso originario, introducendo al contempo una maggiore capacità di adattamento in presenza di model mismatch.

RUN6 realizza quindi un compromesso tra la struttura dell'ottimizzazione convessa e la flessibilità dell'apprendimento profondo, ottenendo una significativa riduzione dell'errore di ricostruzione rispetto alla baseline.

Capitolo 5

Analisi Comparativa e Discussione dei Risultati

In questo capitolo viene presentata un'analisi comparativa sistematica dei diversi approcci implementati (da RUN1 a RUN6), con l'obiettivo di evidenziare il ruolo del model mismatch e l'impatto progressivo dei diversi approcci Model-based Deep Learning.

5.1 Sintesi dei Risultati

La tabella seguente riassume le metriche principali ottenute sui diversi approcci. RUN1 è riportato separatamente perché valutato sul dataset ideale (matched), mentre RUN2–RUN6 sono in regime mismatch.

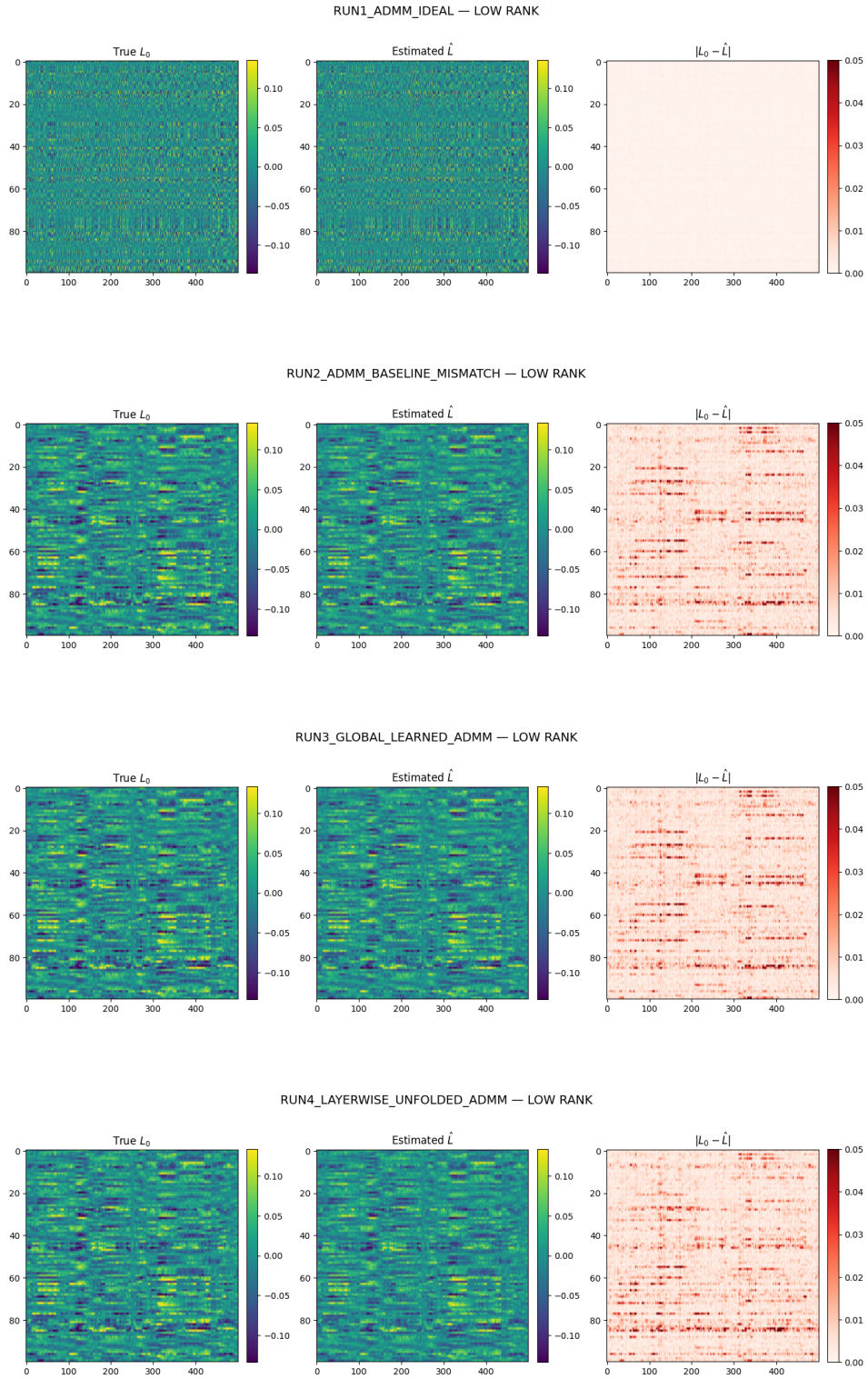
Run	NMSE _L	NMSE _S	Iter / Layer
RUN1 (Ideal)	0.00058	0.0235	433
RUN2 (Mismatch - Baseline)	0.0538	0.822	188
RUN3 (Global Learned Hyperparameters)	0.0538	0.822	95
RUN4 (Unfolded ADMM Layer-wise)	0.0521	0.425	10
RUN5 (Unfolded ADMM + Learned Prox)	0.0356	0.282	10
RUN6 (Unfolded Hybrid Residual ADMM)	0.0191	0.121	10

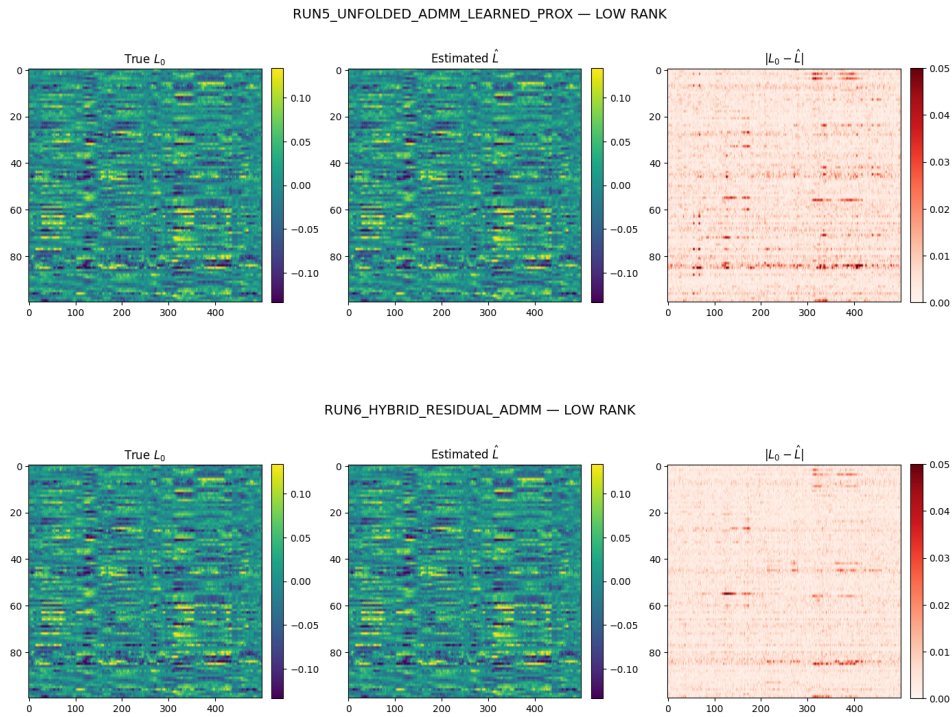
Il confronto tra i vari approcci evidenzia alcuni punti chiave.

- **Impatto del model mismatch.** Il passaggio da RUN1 (dataset ideale) a RUN2 (dataset mismatch) evidenzia un forte deterioramento: NMSE_L passa da 0.00058 a 0.0538, mentre NMSE_S da 0.0235 a 0.822. Il mismatch colpisce entrambe le componenti: le anomalie strutturate e correlate non sono ben modellate da una penalizzazione ℓ_1 elemento per elemento, mentre la variazione temporale del sottospazio e le perturbazioni low-rank locali aumentano il rango effettivo, limitando l'efficacia della sola penalizzazione nucleare.

- **Limiti del tuning globale.** RUN3 dimostra che l'ottimizzazione offline dei parametri (λ, ρ) riduce il numero di iterazioni (da 188 a 95), ma non migliora le metriche di ricostruzione. Il problema non è quindi puramente parametrico e globale, ma legato alla rigidità strutturale del modello.
- **Effetto dell'unfolding layer-wise.** RUN4 introduce parametri learnable per layer con profondità fissata $K = 10$. Si osserva una riduzione marcata di NMSE_S (da 0.822 a 0.425), a parità di struttura proximal analitica. L'adattamento dinamico dei parametri lungo le iterazioni migliora la capacità di compensare il mismatch.
- **Sostituzione del proximal sparse.** RUN5 sostituisce il soft-thresholding con un operatore learnable. Si ottiene un ulteriore miglioramento ($\text{NMSE}_S = 0.282$, $\text{NMSE}_L = 0.0356$), grazie alla capacità della CNN di modellare correlazioni spazio-temporali non catturabili dalla sola norma ℓ_1 .
- **Soluzione migliore: Unfolded Hybrid Residual ADMM.** RUN6 pur preservando la struttura (SVT per la componente low-rank, soft-thresholding per la componente sparse, aggiornamento duale), ogni aggiornamento è arricchito dalla correzione residua learnable appresa dai dati. In questo modo il modello introduce una maggiore capacità di adattamento in presenza di model mismatch e risulta essere l'approccio che ottiene i risultati migliori di ricostruzione.
- **Trade-off tra algoritmo iterativo e apprendimento.** Gli approcci unfoldati (RUN4–RUN5–RUN6) mostrano un miglioramento sostanziale della qualità di ricostruzione e una drastica riduzione del numero di iterazioni effettive (da centinaia a $K = 10$ layer). Tuttavia, tale vantaggio non è gratuito: richiede una fase di addestramento supervisionato sul dataset di training, computazionalmente più onerosa rispetto all'esecuzione di ADMM classico. Il costo viene quindi spostato dalla fase di inferenza alla fase di training: una volta addestrato, il modello fornisce ricostruzioni rapide e più accurate; prima dell'addestramento, però, è necessario un investimento computazionale significativo. Il compromesso risulta vantaggioso quando il modello deve essere applicato ripetutamente a dati con statistiche simili, mentre può essere meno conveniente in scenari con distribuzioni altamente variabili o con disponibilità limitata di dati di training.

5.2 Analisi della ricostruzione della componente low-rank





Le figure riportano, per ciascun approccio sperimentale, la componente low-rank reale L_0 , la stima $\hat{L}$ e la mappa dell'errore assoluto $|L_0 - \hat{L}|$. L'evoluzione visiva lungo i diversi approcci risulta coerente con quanto osservato nelle metriche quantitative.

Nel caso ideale (RUN1), la stima è praticamente sovrapponibile alla matrice reale: l'errore è quasi nullo e distribuito in modo uniforme. Questo conferma che, quando le ipotesi del modello sono soddisfatte, ADMM è in grado di ricostruire correttamente il sottospazio globale.

Con l'introduzione del mismatch (RUN2), la ricostruzione peggiora in modo evidente. La struttura stimata appare meno aderente a quella reale e la mappa dell'errore evidenzia regioni localizzate più marcate. Questo comportamento riflette la violazione dell'ipotesi di low-rank globale statico: il sottospazio varia nel tempo e presenta perturbazioni locali, mentre il modello impone una struttura più rigida.

RUN3 non mostra differenze qualitative sostanziali rispetto a RUN2: il tuning globale dei parametri migliora la velocità di convergenza, ma non modifica in modo significativo la qualità della soluzione finale.

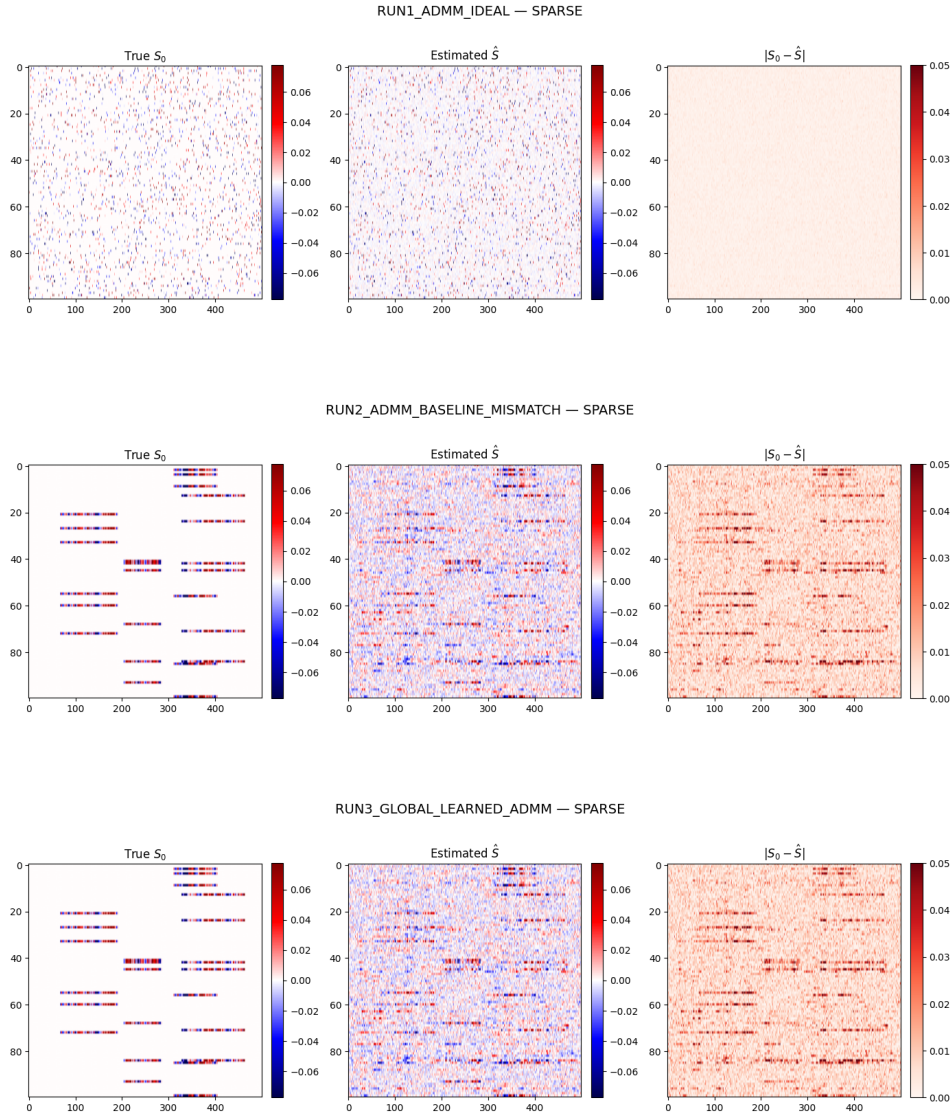
Con RUN4 si osserva un primo miglioramento: l'unfolding layer-wise introduce maggiore flessibilità nella dinamica iterativa, riducendo parzialmente l'intensità e la concentrazione degli errori. Tuttavia, la penalizzazione nucleare rimane invariata e quindi la capacità di adattarsi a sottospazi tempo-varianti resta limitata.

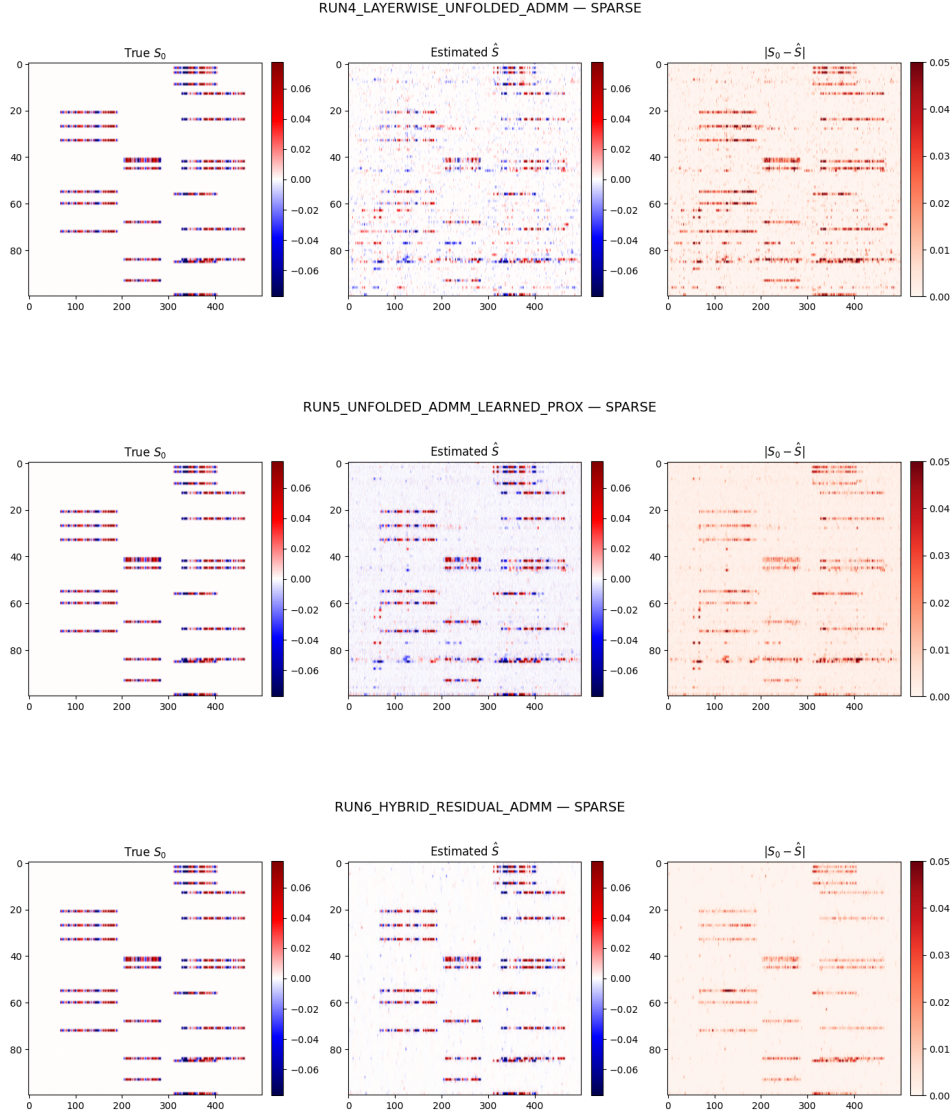
Il miglioramento diventa più evidente in RUN5 e soprattutto in RUN6. In RUN5, una migliore stima della componente sparse si riflette indirettamente anche su L , grazie a una separazione più accurata tra le due componenti. In

RUN6, le correzioni residue learnable permettono invece di adattare direttamente la stima low-rank, riducendo in modo significativo le regioni ad alto errore e rendendo la ricostruzione visivamente più fedele alla struttura reale.

Nel complesso, la sequenza delle figure mostra chiaramente la transizione da un modello convesso rigido, efficace solo in regime ideale, a una dinamica progressivamente più adattiva, capace di compensare il mismatch strutturale presente nei dati.

5.3 Analisi della ricostruzione della componente sparsa





Le figure riportano, per ciascun approccio, la componente sparsa reale S_0 , la stima $\hat{S}$ e la differenza assoluta $|S_0 - \hat{S}|$.

Nel caso ideale (RUN1), la ricostruzione risulta accurata: la struttura sparsa generata casualmente viene recuperata in modo coerente dal soft-thresholding, e la mappa dell'errore è quasi uniforme e di bassa intensità.

Con l'introduzione del mismatch (RUN2), la situazione cambia drasticamente. Le anomalie, ora strutturate in blocchi spazio-temporali persistenti, non sono più ben descritte da una penalizzazione ℓ_1 elemento per elemento. La stima $\hat{S}$ appare fortemente contaminata da rumore diffuso e l'errore evidenzia regioni estese in cui la struttura sparsa non viene correttamente isolata.

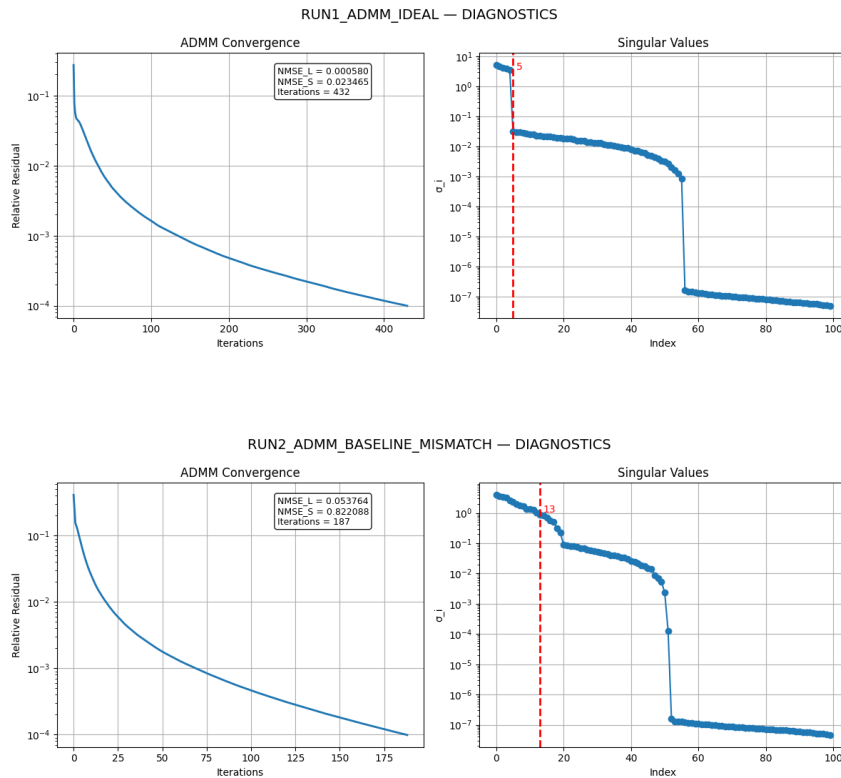
RUN3 non modifica sostanzialmente questo comportamento: la semplice ottimizzazione globale dei parametri accelera la convergenza ma non migliora la qualità della ricostruzione, confermando che il limite è strutturale e non solo parametrico.

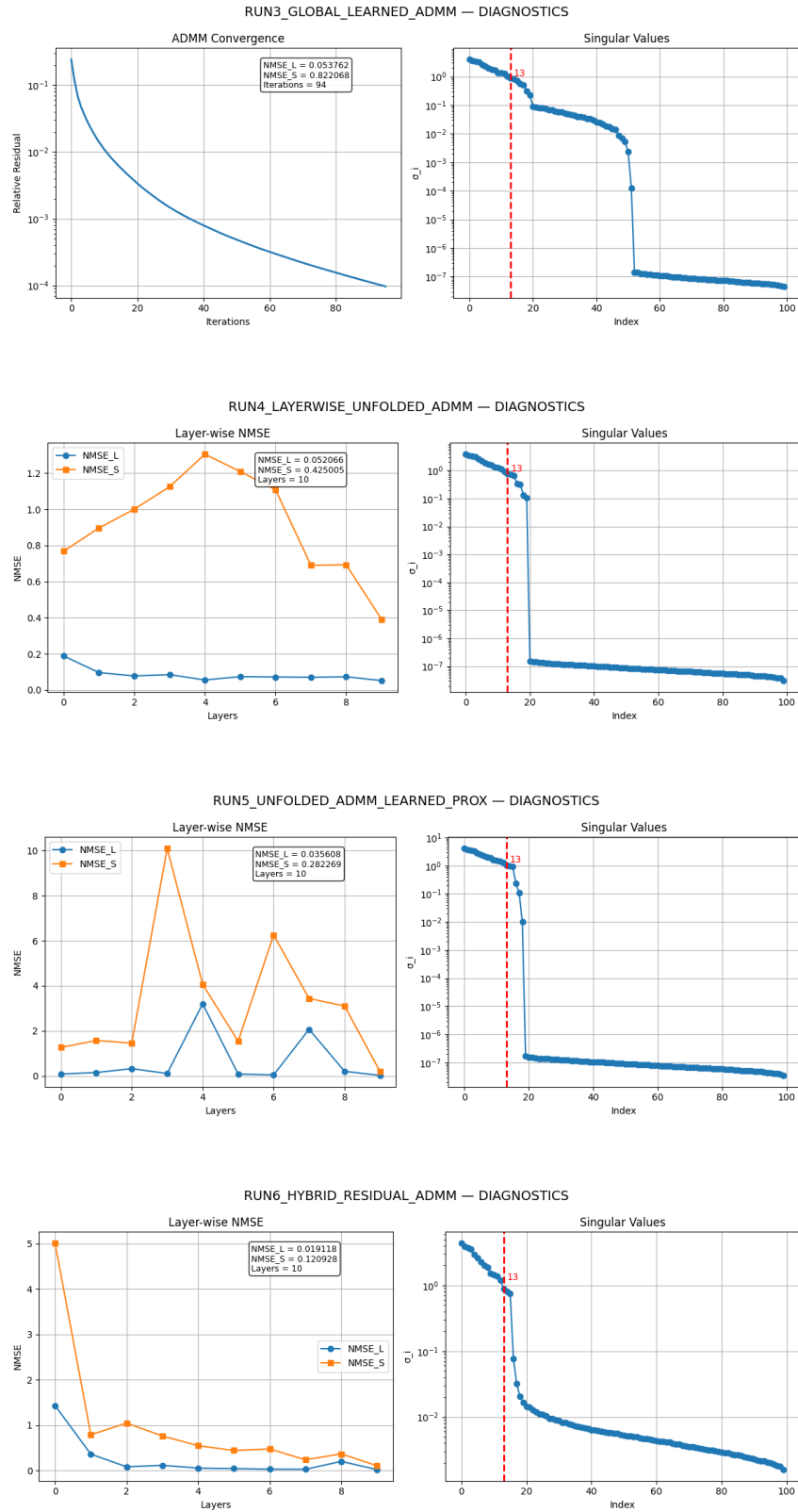
Con RUN4, l'unfolding layer-wise introduce un primo miglioramento visibile: le anomalie principali vengono recuperate con maggiore chiarezza e il rumore spurio si riduce, anche se la ricostruzione rimane ancora parzialmente diffusa.

RUN5 mostra un salto qualitativo più evidente. La sostituzione del proximal analitico con un operatore convoluzionale learnable permette di catturare le correlazioni locali delle anomalie strutturate. Le regioni attive risultano più definite e l'errore si concentra maggiormente lungo i bordi delle strutture, anziché diffondersi sull'intera matrice.

Infine, RUN6 fornisce la ricostruzione più fedele. Le anomalie vengono isolate con maggiore precisione, il rumore residuo è drasticamente ridotto e la mappa dell'errore mostra intensità molto inferiori rispetto ai RUN precedenti. L'integrazione di correzioni residue su entrambe le componenti consente una migliore separazione tra sottospazio nominale e deviazioni sparse, confermando visivamente quanto osservato nelle metriche quantitative.

5.4 Analisi della convergenza e dei valori singolari





Queste figure permettono di analizzare in modo più approfondito il comportamento dinamico dei diversi approcci, sia in termini di convergenza iterativa

(o layer-wise), sia rispetto alla struttura spettrale della componente low-rank stimata.

Convergenza dell'ADMM classico (RUN1–RUN2–RUN3)

Nel caso ideale (RUN1), il residuo relativo decresce in modo regolare e monotono fino alla soglia di arresto, riflettendo un comportamento coerente con le ipotesi teoriche del modello. Nel caso mismatch (RUN2), la convergenza è più rapida ma verso una soluzione subottima: il numero di iterazioni si riduce, ma le metriche finali risultano significativamente peggiori. RUN3 conferma che il tuning globale di (λ, ρ) accelera la convergenza, senza modificare in modo sostanziale la qualità della soluzione raggiunta.

Dinamica layer-wise negli approcci unfolded (RUN4–RUN5–RUN6)

Negli approcci unfolded, l'analisi layer-wise mostra come l'errore non decresca necessariamente in modo monotono: i layer iniziali possono temporaneamente aumentare l'NMSE per poi ridurlo negli strati successivi. Questo comportamento evidenzia che la rete non replica semplicemente l'algoritmo iterativo classico, ma apprende una traiettoria ottimizzata a profondità finita. In RUN5 tale fenomeno è particolarmente marcato (picchi NMSE > 10): l'operatore convoluzionale inizializzato casualmente produce stime di S perturbate che si propagano su L attraverso il termine $Y - S^k + U^k$, amplificando l'errore prima che la rete impari a correggersi. In RUN6, l'inizializzazione $\alpha = 0.05$ nei blocchi residuali smorza le perturbazioni iniziali, producendo una riduzione più stabile dell'NMSE negli ultimi layer.

Analisi dei valori singolari

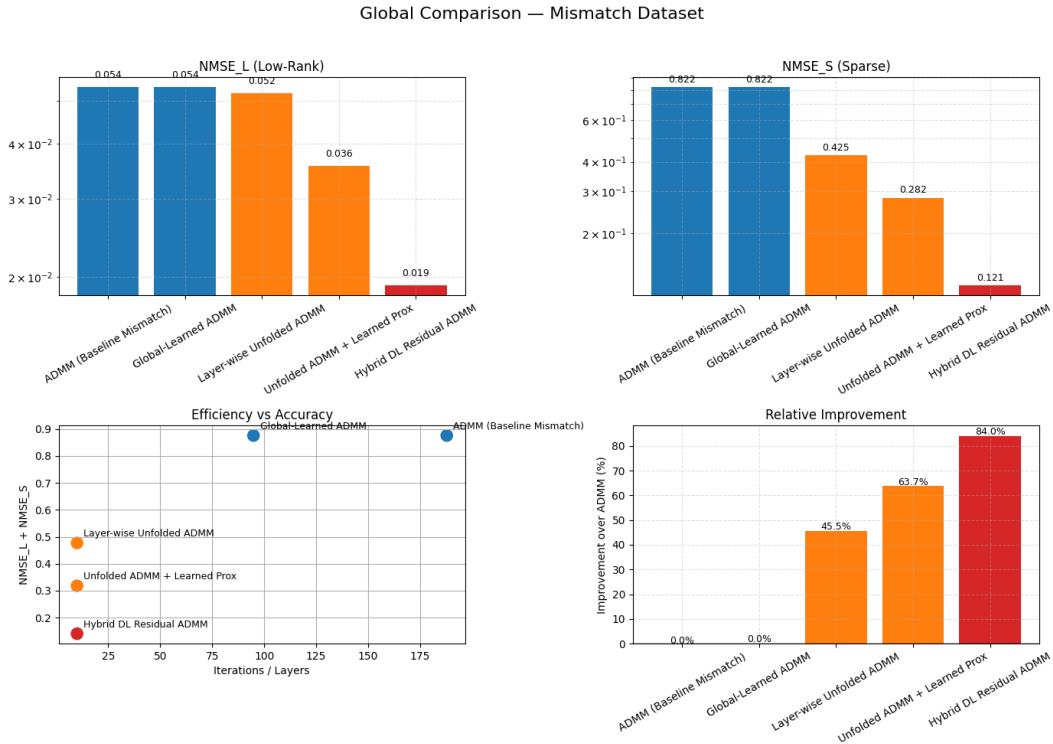
L'analisi spettrale della componente stimata $\hat{L}$ consente di valutare quanto ciascun approccio riesca a ricostruire correttamente la struttura di rango dei dati.

Nel dataset ideale (RUN1), il rango effettivo è pari a $r = 5$. Lo spettro mostra infatti una chiara separazione tra i primi cinque valori singolari dominanti e i successivi, che risultano prossimi a zero. Il leggero residuo non nullo è dovuto alla presenza di rumore numerico e alla tolleranza di arresto dell'algoritmo, ma la distinzione tra sottospazio informativo e componente residua risulta netta.

Nel dataset mismatch, il rango effettivo aumenta a circa 13, a causa della variazione temporale del sottospazio e delle perturbazioni low-rank locali introdotte nella generazione dei dati. Nel caso RUN2 e RUN3 lo spettro presenta un numero maggiore di valori singolari significativi. Gli approcci basati solo sulle assunzioni del modello tendono a comprimere lo spettro tramite la penalizzazione nucleare, ma la separazione tra componenti dominanti e coda spettrale risulta meno marcata.

Negli approcci data-driven invece (RUN4–RUN5–RUN6), la distribuzione spettrale rispecchia più fedelmente il rango effettivo del dataset mismatch. In particolare, i primi ≈ 13 valori singolari risultano chiaramente distinti dalla parte residua dello spettro, mentre i valori successivi si collocano in una regione molto più prossima allo zero (pur non essendo esattamente nulli a causa di rumore e approssimazioni numeriche). RUN6 comunque mostra la separazione più netta tra componenti significative e residuo spettrale, indicando una migliore capacità di adattamento alla reale dimensionalità del sottospazio latente.

5.5 Confronto Globale: Accuratezza ed Efficienza



Questa ultima figura conclude la parte di analisi sperimentale, fornendo una sintesi comparativa complessiva degli approcci in regime di model mismatch. I grafici superiori evidenziano la riduzione progressiva di $NMSE_L$ e $NMSE_S$ al crescere dell'integrazione tra modello e apprendimento. Il diagramma efficienza-accuratezza mostra chiaramente il vantaggio degli approcci unfolded, che raggiungono errori inferiori con un numero di layer limitato. Infine, l'analisi del miglioramento relativo quantifica in modo diretto il guadagno rispetto alla baseline, evidenziando come RUN6 rappresenti il miglior compromesso tra precisione e profondità computazionale.

Capitolo 6

Conclusioni e Sviluppi Futuri

Conclusioni

Il lavoro ha analizzato il problema della decomposizione di matrici di traffico Internet secondo l'assunzione del modello low-rank + sparse:

$$Y = L_0 + S_0 + N,$$

studiando in modo progressivo il passaggio da un approccio puramente basato sulle assunzioni del modello tramite ADMM a formulazioni unfoldate con componenti learnable, riconducibili al paradigma del Model-Based Deep Learning.

L'analisi sperimentale, condotta su dataset sintetici controllati, ha evidenziato in modo chiaro l'impatto del model mismatch. In condizioni ideali, l'ADMM classico garantisce una ricostruzione quasi perfetta. In presenza di sottospazi tempo-varianti, perturbazioni locali e anomalie strutturate, le prestazioni si degradano significativamente, mostrando i limiti della sola penalizzazione nucleare e ℓ_1 .

Il semplice tuning globale degli iperparametri migliora la velocità di convergenza ma non la qualità della soluzione, confermando che la limitazione è strutturale e non solo parametrica.

L'introduzione del deep unfolding ha invece permesso di apprendere una dinamica iterativa più adattiva, migliorando sensibilmente la ricostruzione in regime mismatch e riducendo drasticamente il numero di iterazioni effettive. La sostituzione dei proximal analitici con operatori convoluzionali learnable ha ulteriormente aumentato la capacità espressiva del modello, consentendo di modellare correlazioni spazio-temporali non catturate dalla formulazione convessa originale.

Nel complesso, i risultati mostrano che l'integrazione tra struttura algoritmica basata su assunzioni del modello e Deep Learning rappresenta una strategia efficace per compensare deviazioni strutturali rispetto alle ipotesi teoriche del modello. In particolare, gli approcci unfoldati si dimostrano par-

ticolarmente adatti a gestire scenari di mismatch, adattando dinamicamente la stima delle componenti latenti.

Un aspetto centrale emerso dal lavoro è il trade-off computazionale: gli approcci unfoldati richiedono una fase di addestramento potenzialmente onerosa e una quantità adeguata di dati per esprimere pienamente la loro capacità adattiva. Tuttavia, una volta addestrati, consentono inferenza rapida a profondità fissa, rendendo il costo per campione significativamente inferiore rispetto a un algoritmo iterativo classico eseguito fino a convergenza. Il bilanciamento tra costo di training, disponibilità di dati e costo di inferenza diventa quindi un elemento chiave in vista di applicazioni operative.

Sviluppi Futuri

Tra le possibili estensioni si evidenziano:

- l'adozione di architetture unfoldate più avanzate, come CORONA (Convolutional rObust pRincipal cOmpoNent Analysis). In pratica, a differenza di RUN6, in cui le correzioni learnable sono additive e agiscono in forma residuale sugli aggiornamenti analitici di SVT e soft-thresholding, CORONA sostituisce integralmente i prossimali della struttura ADMM con layer convoluzionali learnable. Le matrici di proiezione implicite negli operatori prossimali non sono più quindi determinate dalla norma nucleare o dalla norma 1, ma vengono parametrizzate direttamente come filtri convoluzionali ottimizzati end-to-end.
- la validazione su dataset reali di traffico Internet, caratterizzati da dinamiche più complesse;
- l'esplorazione di moduli deep più espressivi mantenendo un nucleo algoritmico interpretabile e coerente con la struttura del problema.

Il paradigma Model-Based Deep Learning si configura quindi come un ponte naturale tra ottimizzazione convessa e apprendimento automatico. Esso consente di preservare la struttura, l'interpretabilità e la stabilità teorica dei metodi model-based, introducendo al tempo stesso la flessibilità necessaria per adattarsi a dati reali caratterizzati da mismatch strutturale. In questa prospettiva, l'integrazione tra algoritmo e apprendimento non rappresenta una semplice estensione tecnica, ma una strategia metodologica capace di coniugare rigore matematico, efficienza computazionale e capacità di generalizzazione.

Riferimenti Bibliografici

Durante la progettazione, la realizzazione del progetto e la stesura della tesina, sono state consultate le seguenti fonti:

- N. Shlezinger, Y. C. Eldar, S. P. Boyd, *Model-Based Deep Learning: On the Intersection of Deep Learning and Optimization*, 2022.
- N. Shlezinger, J. Whang, Y. C. Eldar, A. G. Dimakis, *Model-Based Deep Learning*, 2021.
- V. Monga, Y. Li, Y. C. Eldar, *Algorithm Unrolling: Interpretable, Efficient Deep Learning for Signal and Image Processing*, 2021.
- H. Brehier, A. Breloy, C. Ren, G. Ginolhac, *Deep unrolling of Robust PCA and Convolutional Sparse Coding for stationary target localization in through wall radar imaging*, 2021.
- E. J. Candès, X. Li, Y. Ma, J. Wright, *Robust Principal Component Analysis?*, Journal of the ACM, 58, 1–37, 2011.
- S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers*, 2011.
- Y. Yang, J. Sun, H. Li, Z. Xu, *ADMM-Net: A Deep Learning Approach for Compressive Sensing MRI*, 2016.
- D. Bertsimas, R. Cory-Wright, N. A. G. Johnson, *Sparse Plus Low Rank Matrix Decomposition: A Discrete Optimization Approach*, 2015.
- O. Solomon, R. Cohen, Y. Zhang, Y. Yang, Q. He, J. Luo, R. G. van Sloun, Y. C. Eldar, *Deep Unfolded Robust PCA With Application to Clutter Suppression in Ultrasound*, IEEE, 2021.